



MST(Minimum Spanning Tree 최소신장 트리) Kruskal 알고리즘

<https://youtu.be/g7mdefM-HqY?feature=shared>

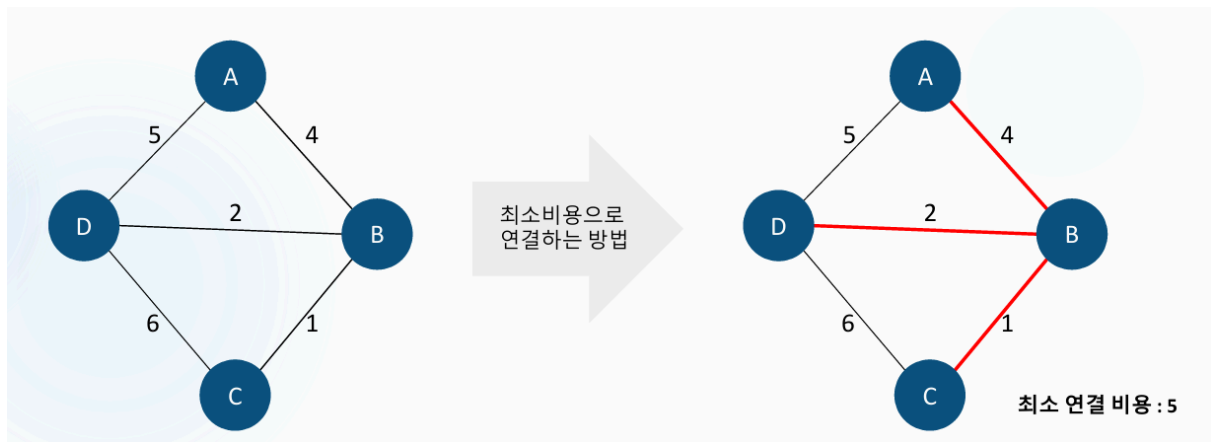
MST 크루스칼 알고리즘 & BFS 길찾기 수업노트

MST: Minimum Spanning Tree

기본 개념

그래프에서 최소비용으로 모든 구간을 연결한 트리

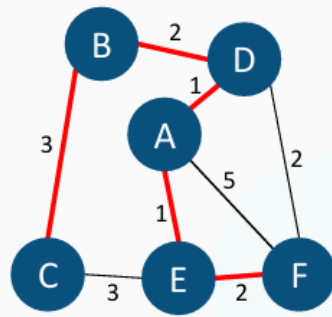
예시 그래프



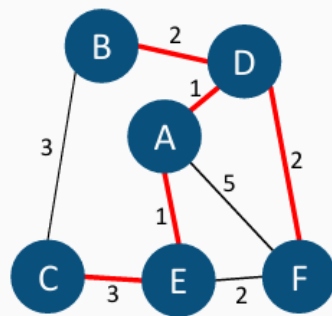
MST의 특징

1. 트리가기 때문에 cycle 존재하면 안된다.
2. 답은 여러개가 될 수 있다.

MST 예시 (복잡한 그래프)



최소 연결 비용 : 9



최소 연결 비용 : 9

KRUSKAL 알고리즘 요약

MST를 구하는 대표적인 알고리즘

시간복잡도는 $O(N \log N)$ 여기서 n은 엣지타는 간선의 수를 뜻한다.

크루스칼 알고리즘 과정

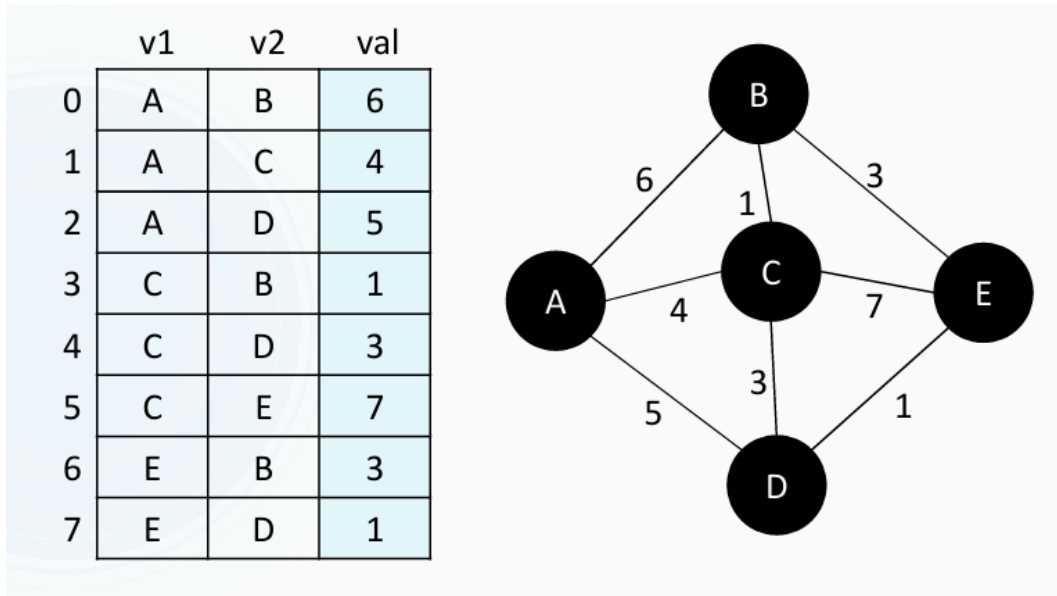
1. 가중치 값을 기준으로 정렬을 먼저 한다.
2. Union Find 자료구조를 이용해서 간선을 선택해준다.

크루스칼 알고리즘 과정 1

그래프를 아래와 같이 하드코딩 해준다.

v1과 v2의 순서는 반대여도 상관없다.

간선 리스트 표현



1. 정렬하기

가중치(val) 기준으로 오름차순 정렬:

	v1	v2	val
0	A	B	6
1	A	C	4
2	A	D	5
3	C	B	1
4	C	D	3
5	C	E	7
6	E	B	3
7	E	D	1

→

	v1	v2	val
0	C	B	1
1	E	D	1
2	E	B	3
3	C	D	3
4	A	C	4
5	A	D	5
6	A	B	6
7	C	E	7

2. 간선선택하기

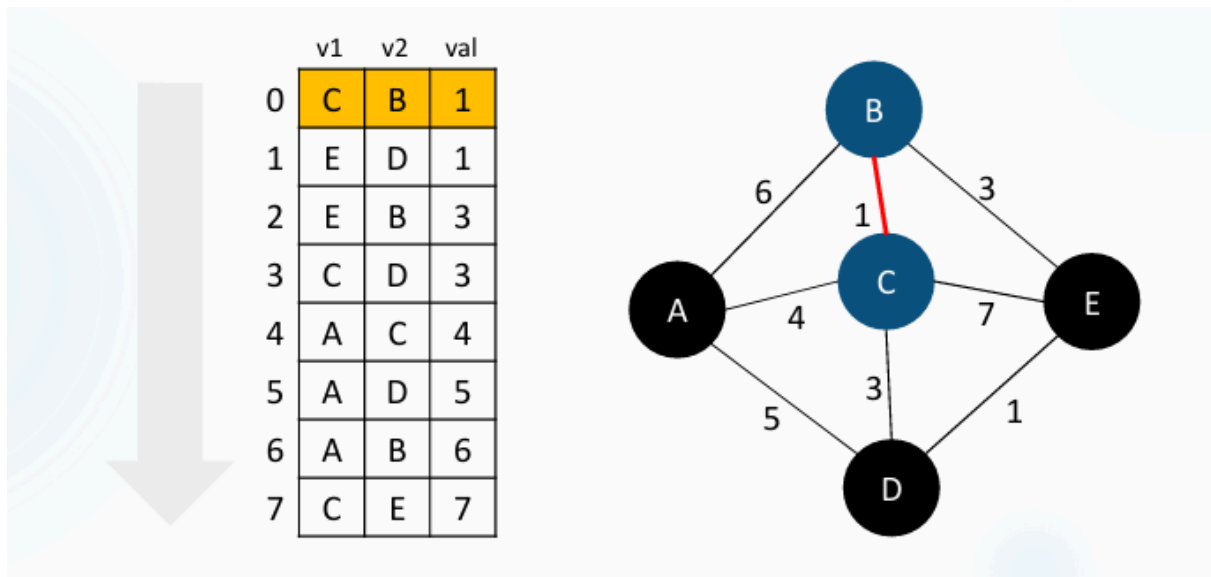
단계별 진행 과정

1순 for문을 돌리면서 MST가 될 간선을 정정짓는다.

맨앞에 있는 C-B간선은 MST 간선으로 채택된다.

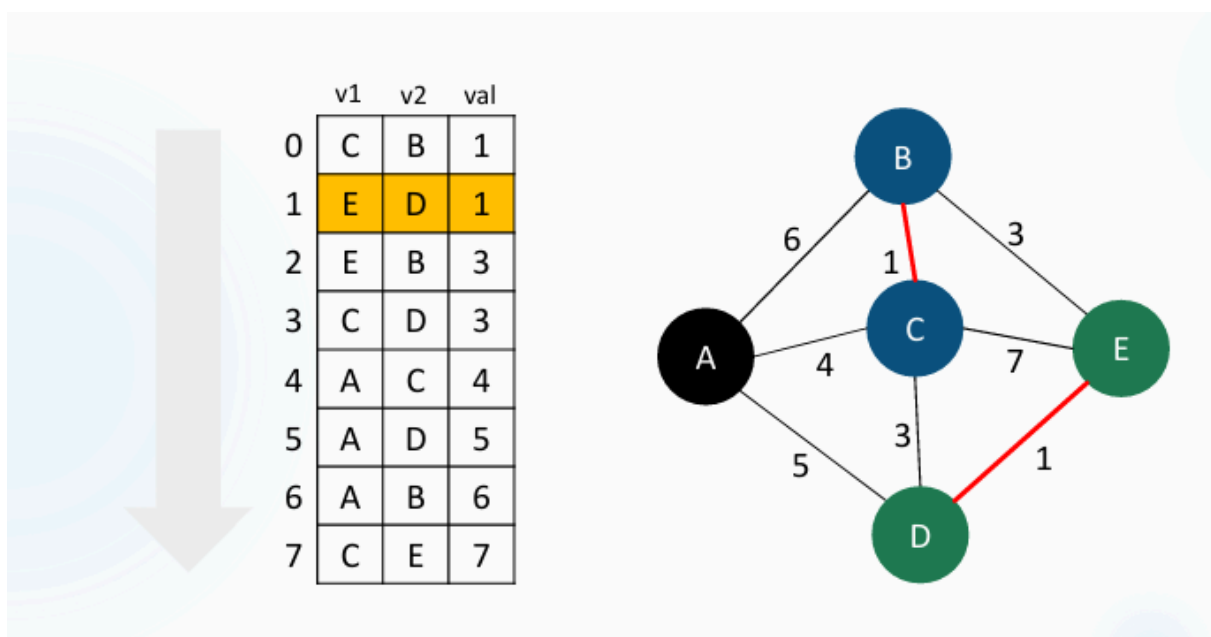
노드 C와 B는 같은 그룹이된다(Union Find알고리즘 사용)

1단계: C-B 간선 선택



C와 B가 같은 그룹으로 연결됨

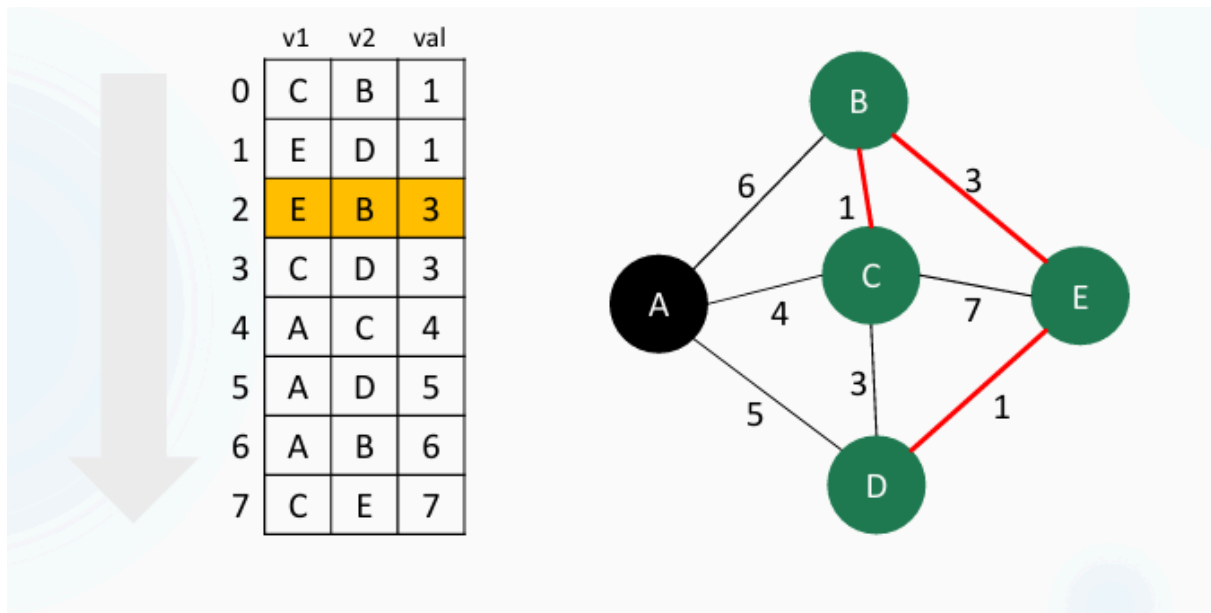
2단계: E-D 간선 확인



E-D 간선이 선택되고, E-D 가 같은 그룹인지 확인한후 아니라면 같은 그룹이 된다.

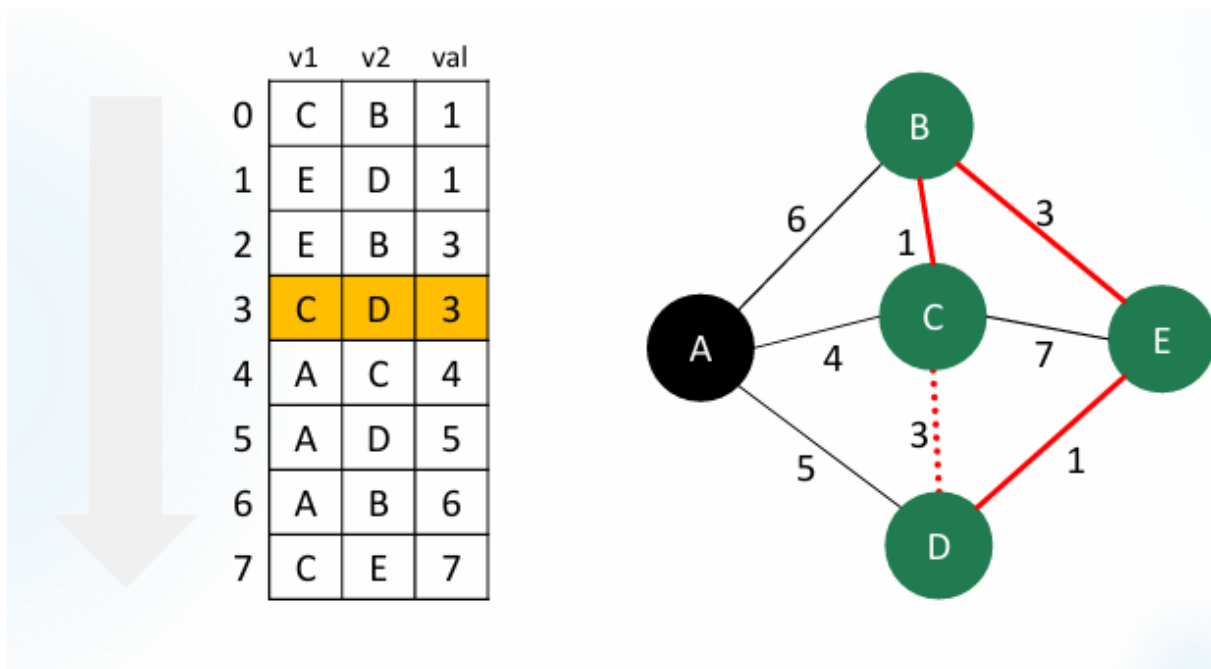
E-D는 간선 채택이 된다.

3단계: E-B 간선 확인



E-B에서 E,B는 각각 다른 그룹이기에 같은 그룹으로 만들고 간선 채택시킨다.

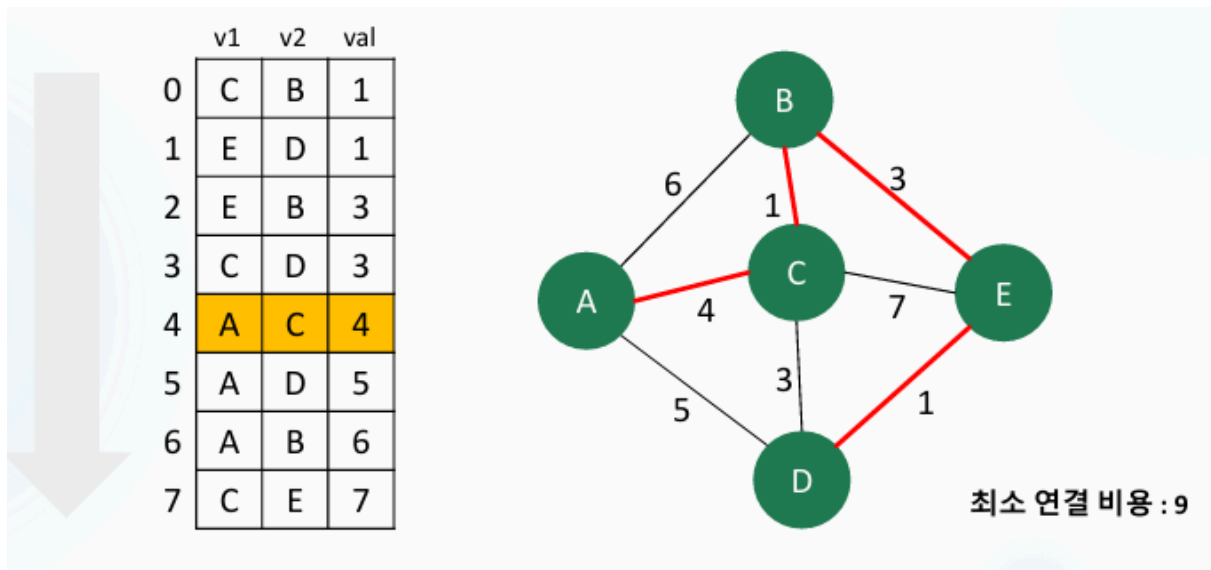
4단계: C-D 간선 확인



C-D는 같은 그룹이다. 같은 그룹인 노드를 연결시키면 Cycle이 발생하게 된다.

간선으로 채택하지 않고 넘어간다.

5단계: A-C 간선 선택



이와 같은 방법으로 나머지 간선정보들을 전부 순회시켜주면 MST가 완성된다.

MST 크루스칼 알고리즘 구현

완전한 소스코드

```
#include <iostream>
#include <algorithm>
#include <vector>

struct Node
{
    char v1;
    char v2;
    int cost;
};

std::vector<Node> graph =
{
    {'A', 'B', 6},
    {'A', 'C', 4},
    {'A', 'D', 5},
    {'C', 'B', 1},
    {'C', 'D', 3},
    {'C', 'E', 7},
}
```

```

        {'E', 'B', 3},
        {'E', 'D', 1}
    };

    int n = 8; //간선의 갯수

    bool compare(Node a, Node b)
    {
        return a.cost < b.cost;
    }

    int org[200] = {};

    int findParent(int now)
    {
        if (org[now] == 0)
        {
            return now;
        }

        int ret = findParent(org[now]);
        org[now] = ret;
        return ret;
    }

    int unionOrg(int v1, int v2)
    {
        int p1 = findParent(v1);
        int p2 = findParent(v2);

        if (p1 == p2)
        {
            return 0;
        }

        org[p1] = p2;
        return 1;
    }

```

```

int main()
{
    std::sort(graph.begin(), graph.end(), compare);

    int cnt = 0;
    int sum = 0;

    for (size_t i = 0; i < graph.size(); i++)
    {
        if (unionOrg(graph[i].v1, graph[i].v2))
        {
            sum += graph[i].cost;
            cnt++;
        }
    }

    if (cnt == 4/*노드의개수 - 1 */)
    {
        std::cout << sum << std::endl;
    }

    return 0;
}

```

핵심 함수 분석

1. compare 함수

```

bool compare(Node a, Node b)
{
    return a.cost < b.cost;
}

```

가중치 기준 오름차순 정렬용 비교함수

2. findParent 함수 (Union-Find)


```

int findParent(int now)
{
    if (org[now] == 0)
    {
        return now;
    }

    int ret = findParent(org[now]);
    org[now] = ret; // 경로 압축
    return ret;
}

```

노드의 루트 부모를 찾는 함수 (경로 압축 최적화)

3. unionOrg 함수

```

int unionOrg(int v1, int v2)
{
    int p1 = findParent(v1);
    int p2 = findParent(v2);

    if (p1 == p2)
    {
        return 0; // 같은 그룹 (사이클 발생)
    }

    org[p1] = p2; // 그룹 병합
    return 1;    // 병합 성공
}

```

두 노드를 같은 그룹으로 병합, 사이클 검사

BFS 길찾기 알고리즘

BFS 기본 특징

BFS(Breadth-First Search)는 그래프나 트리 구조에서 가장 가까운 노드부터 탐색하는 방식입니다.

- 큐(Queue) 자료구조를 사용합니다.
- 최단 경로를 찾는 데 유용합니다.
- 시간복잡도는 $O(V+E)$ 입니다. (V: 정점의 개수, E: 간선의 개수)

BFS 길찾기 구현

```
#include <iostream>
#include <queue>
#include <vector>
#include <tuple>
using namespace std;

const int dx[] = { -1, 1, 0, 0 };
const int dy[] = { 0, 0, -1, 1 };

//s -> 시작 좌표, e -> 도착 좌표
int bfs(const vector<vector<int>>& map, int sx, int sy, int
ex, int ey)
{
    int N = map.size();
    int M = map[0].size();
    vector<vector<bool>> visited(N, vector<bool>(M, fals
e));
    vector<vector<int>> dist(N, vector<int>(M, -1)); // 거
리 배열

    queue<pair<int, int>> q;
    q.push({ sx, sy });
    visited[sx][sy] = true;
    dist[sx][sy] = 0;

    while (!q.empty())
    {
        auto x = q.front().first;
        auto y = q.front().second;
        q.pop();

        // 도착지 도달
```

```

        if (x == ex && y == ey)
            return dist[x][y];

        for (int dir = 0; dir < 4; ++dir)
        {
            int nx = x + dx[dir];
            int ny = y + dy[dir];

            // 맵 내부 + 벽이 아님 + 미방문
            if (nx >= 0 && nx < N && ny >= 0 && ny < M)
            {
                if (!visited[ny][nx] && map[ny][nx] == 0)
                {
                    visited[ny][nx] = true;
                    dist[ny][nx] = dist[y][x] + 1;
                    q.push({ nx, ny });
                }
            }
        }

        return -1; // 도달 불가능
    }
}

```

BFS 알고리즘 특징

핵심 요소

- **visited 배열**: 방문 체크로 중복 방문 방지
- **dist 배열**: 각 위치까지의 최단 거리 저장
- **4방향 탐색**: dx, dy 배열로 상하좌우 이동
- **큐 사용**: FIFO 구조로 레벨별 탐색

동작 과정

1. 시작점을 큐에 추가하고 방문 처리
2. 큐가 빌 때까지 반복:

- 큐에서 현재 위치 꺼내기
- 목표 도달시 거리 반환
- 4방향으로 이동 가능한 곳 탐색
- 조건 만족시 큐에 추가

장점과 활용

- **최단 경로 보장**: 레벨별 탐색으로 최단 거리 찾기
- **격자 맵 탐색**: 게임, 로봇 경로 계획
- **미로 찾기**: 출구까지의 최단 경로

MST vs BFS 비교

구분	MST (크루스칼)	BFS 길찾기
목적	최소 비용 연결	최단 경로 탐색
자료구조	Union-Find	Queue
시간복잡도	$O(E \log E)$	$O(V + E)$
활용분야	네트워크 설계	경로 탐색
결과	최소 신장 트리	최단 거리

실무 활용 사례

MST 활용

- **네트워크 설계**: 최소 비용으로 모든 도시 연결
- **클러스터링**: 데이터 그룹핑
- **회로 설계**: 최소 배선으로 모든 부품 연결

BFS 활용

- **게임 AI**: 캐릭터 최단 이동 경로
- **네비게이션**: 최단 거리 길찾기
- **소셜 네트워크**: 친구 관계의 최단 연결

핵심 포인트

MST 크루스칼 알고리즘

1. 간선 정렬: 가중치 기준 오름차순
2. Union-Find: 사이클 검사 및 그룹 관리
3. 간선 선택: 사이클 없이 최소 비용
4. 완성 조건: (노드 수 - 1)개의 간선

BFS 길찾기

1. 큐 활용: FIFO로 레벨별 탐색
2. 방문 체크: visited 배열로 중복 방지
3. 거리 기록: dist 배열로 최단 거리 저장
4. 4방향 탐색: 상하좌우 이동 처리

두 알고리즘 모두 그래프 문제 해결의 핵심 도구입니다.

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,
- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

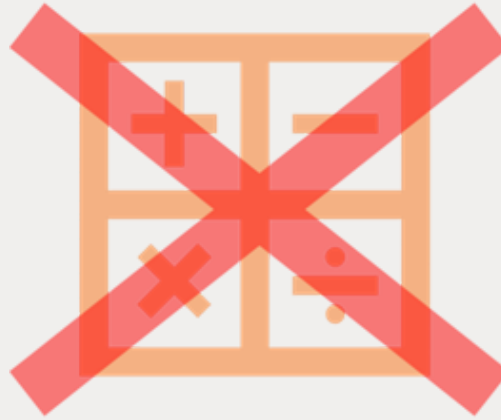
▼ oopy:hide

연습문제



문제 1번

수학이는 일반적인 사칙연산이 마음에 들지 않았습니다.
자신만의 사칙연산을 만들어 세상에 널리 알리고 싶어합니다.



연산자	설명	예시
!!	2회 더한다.	$1 !! 3 = 7$
#	2회 뺀다.	$5 \# 1 = 2$
\$	무조건 10을 더한다.	$3 \$ 8 = 13$
&	두번째 숫자 2승 후 더한다.	$2 \& 3 = 11$
^	답은 무조건 0	$3 ^ 7 = 0$

수학이는 자신이 만든 연산자를 사용해야하는 빈칸 채우기 문제를 냈습니다.
숫자1 [] 숫자2 [] 숫자3 ≥ 20

숫자 3개를 입력받고, 숫자 3개 사이에 2개의 연산자를 사용합니다.
이렇게 구해진 수가 20보다 큰 수를 찾는 수식입니다.
우리는 5개 연산자 중 2개를 선택하여, 조건을 만족 시켜야합니다.
조건을 만족시킬 수 있는 총 경우의 수를 출력 해 주세요.
(Backtracking을 사용하여 문제를 풀어주세요.)

[입력]

숫자1, 2, 3을 입력 받으세요.

[출력]

결과 값이 20이 이상이 되는 경우의 수를 출력하세요.

[예제]

1 5 2 를 입력 받았을때 !! 와 \$ 연산자를 사용하면

$1 !! 5 \$ 2 = 21$ 이 되고, 이는 20보다 크므로 조건을 만족시킬 수 있습니다.

입력 예제

1 5 2

출력 결과

6

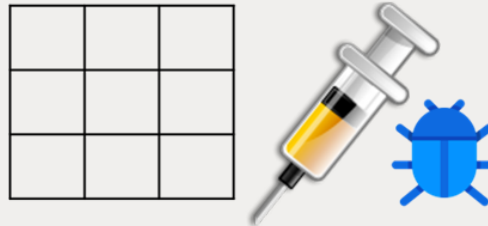


문제 2번

바이러스를 투여하고, 바이러스가 모두 퍼지고 난 뒤의 결과를 출력해주세요.

바이러스는 두 곳에 투여되며, 상, 하, 좌, 우 네 방향으로 퍼지게 됩니다.

바이러스가 퍼지는 맵의 크기는 3 X 3 입니다.



[입력]

바이러스가 투여되는 좌표 Y X 두곳을 입력하세요.

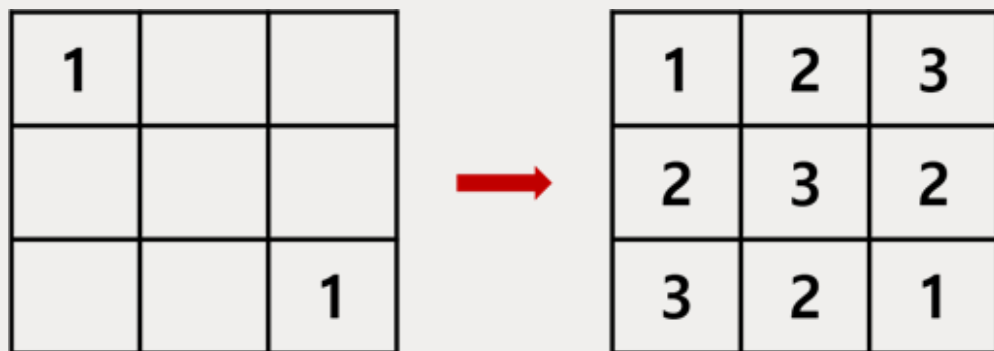
[출력]

바이러스가 모두 퍼지고 난 뒤의 상태를 출력해주세요.

[예시 1]

(0, 0) 과 (2, 2) 에 바이러스를 투여한다면 아래처럼 바이러스는 상하좌우 네 방향으로 퍼지며, 강도는 1씩 증가합니다.

(0, 0) 과 (2, 2) 에서 바이러스가 모두 퍼지면 위와 같은 결과가 출력되어야 합니다.



[예시 2]

만약 (2, 1) (2, 2)에 바이러스를 투여한다면 아래와 같은 결과가 출력되어야 합니다.

	1	1



4	3	3
3	2	2
2	1	1

입력 예제

0 0

2 2

출력 결과

123

232

321



문제 3번

시작 좌표에서 종료 좌표까지 최소 몇 회만에 갈 수 있는지 확인해주세요.

BFS 알고리즘을 사용하여 길을 찾아주세요.

아래 1의 위치로는 이동할 수 없습니다.

2차원 배열의 맵은 하드코딩하여 풀어 주세요.

1	1		1
1		1	

[입력]

시작 좌표 Y X 와, 그 다음 줄에 종료 좌표 Y X 를 입력하세요.

[출력]

시작 좌표 부터 종료 좌표까지 최소 몇번 만에 도착할 수 있는지 출력해주세요.

[예시 1]

시작 좌표 (0, 0) 과 도착 좌표 (3, 3) 이 입력 될때, 결과는 6회입니다.

!			
1	1		1
1		1	!

입력 예제

0 0

3 3

출력 결과

6회



문제 4번

쥐 			친구 	
				

(0, 0)에 서있는 쥐는 **치즈를 먼저 먹고**, 친구를 만나러 가려고 합니다.

쥐는 $\uparrow \downarrow \leftarrow \rightarrow$ 로 이동 할 수 있습니다.

[입력]

치즈 위치 (Y, X) 와 친구 좌표 (N, M) 을 입력 받으세요.

[출력]

친구를 만나기까지의 최단 거리를 출력 해주세요.

한 칸을 이동했을때가 1입니다.

[예시]

위 예제에서는 **9회** 이동으로 치즈를 먹고 친구를 만날 수 있습니다.

입력 예제

2 0

0 3

출력 결과

9



문제 5번

4 × 6 사이즈의 맵에 있는 모든 치킨을 먹으려고 합니다.

지혁이는 [0, 0]에서 출발하고, ↑ ↓ ← → 방향으로 이동할 수 있습니다.

0	0	1		1	
1	0	0	1	0	1
0	0	1		0	0
	0	0	0	0	0

1은 벽이라 이동할 수 없습니다.

2는 고기입니다.

0으로 표시된 지역만 이동할 수 있습니다.

[입력]

4 X 6 의 2차원 배열 정보를 입력 받으세요.

[출력]

치킨을 총 몇 마리 먹을 수 있는지 출력하세요.

입력 예제

0 0 1 2 1 2

1 0 0 1 0 1

0 0 1 2 0 0

2 0 0 0 0 0

출력 결과

2



문제 6번

현준이가 설치한 폭죽은 1초당 한번씩 불꽃이 사방으로 퍼져나갑니다.

폭죽이 한번 터질 때 마다 8방향 으로 터져 나가고, 연달아 8 방향

(←↖↑↗→↘↓↙)으로 또 터집니다.

만약 0초에 폭죽이 1개 있다면,

1초에는 폭죽이 8개 불꽃으로 터집니다.

2초에는 8개 불꽃이 각각 8개씩 64개로 퍼집니다.

3초에는 64개 불꽃은 $64 \times 8 = 512$ 개로 퍼집니다.

먼저 4×5 사이즈의 맵을 입력 받으세요.

0은 빈 하늘을, 1은 폭죽을 의미합니다.

[입력]

4×5 사이즈의 맵을 입력 하세요.

[출력]

하늘 전체를 불꽃으로 채우는데 걸리는 시간을 출력 하주세요.

[예시 1]

0	0		0	
0	0	0	0	0
0		0	0	0
0	0	0	0	0

[1초 경과]

0				
				
			0	0
			0	0

[2초 경과]

				
				
				
				0

[3초 경과]

예시 1에서 하늘을 불꽃으로 모두 채우는데 걸리는 시간은 3초 입니다.

입력 예제

0 0 1 0 1

0 0 0 0 0

0 1 0 0 0

0 0 0 0 0

출력 결과

3



문제 7번

지효는 땅이 얼마나 큰지 알아내는 드론을 띄웠습니다.

↑ ↓ ← → 으로 땅이 붙어있는 경우는 같은 땅입니다.



[입력]

4×4 땅 상태를 입력 받아 주세요.

 1	 1	0	 1
0	 1	0	 1
0	 1	 1	0
 1	0	0	0

[출력]

(0, 0) 부터 상, 하, 좌, 우 네 방향으로 붙어있는 땅의 총 개수를 출력해주세요.

입력 예제

1 1 0 1

0 1 0 1

0 1 1 0

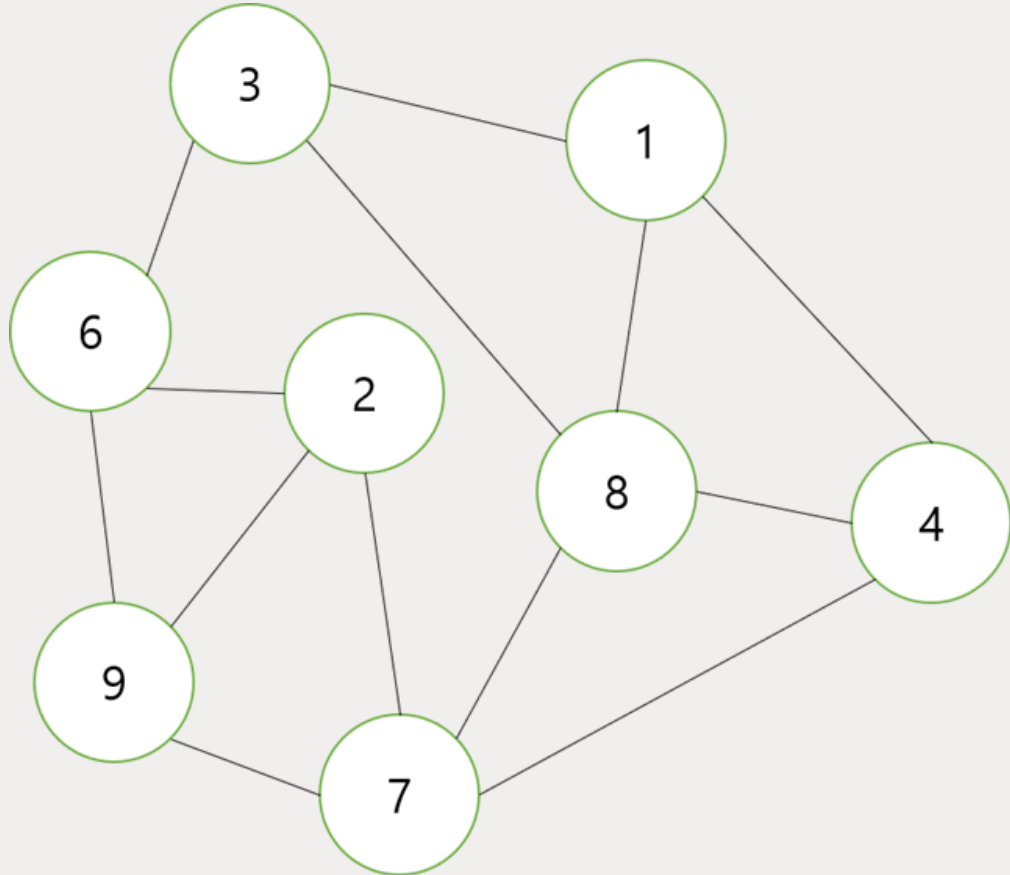
1 0 0 0

출력 결과



문제 8번

아래와 같은 숫자 노드 그래프를 인접 행렬로 하드 코딩해주세요.
그리고 입력받은 노드부터 BFS를 이용하여 노드의 값을 출력해주세요.



BFS 알고리즘을 사용하여 탐색했을때, 작은 숫자의 노드부터 탐색하기 때문에

출발지가 3인 예제의 답은 **3 1 6 8 4 2 9 7** 입니다.

[입력]

출발지를 입력 받으세요.

[출력]

BFS 로 탐색하면서 순서대로 노드 값을 출력하세요.

입력 예제

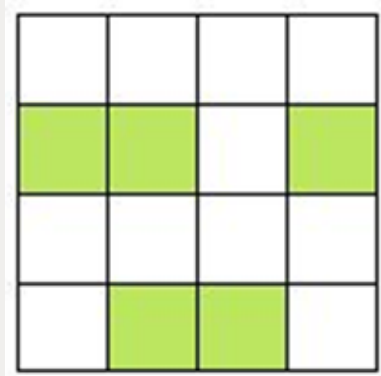
3

출력 결과

3 1 6 8 4 2 9 7



문제 9번



위의 맵배열을 하드코딩 한 후,
시작 위치와 도착지점을 입력 받으세요. (y, x)
최소 몇칸만에 갈 수 있는지 DFS를 돌려 답을 구해주세요.

입력 예제

0 0

3 3

출력 결과

6

DFS 응용 문제들을 풀어봅니다. DFS 문제들을 설계 할 때는 다음과 내용들을 미리 정의 해 둡시다.

1. 트리 형태를 그려, Branch / Level이 몇 개인지 명확히 명시하기
2. 바닥조건 / 가지치기 조건을 명확히 하기
3. 추가 함수를 구현하는 것이 더 좋을 지 생각 해 보기
4. 원상복구를 어떻게 구현할지
5. 함수 Interface 결정



문제 1번

3 × 3 숫자판에서 3개의 숫자를 선택해야 합니다.

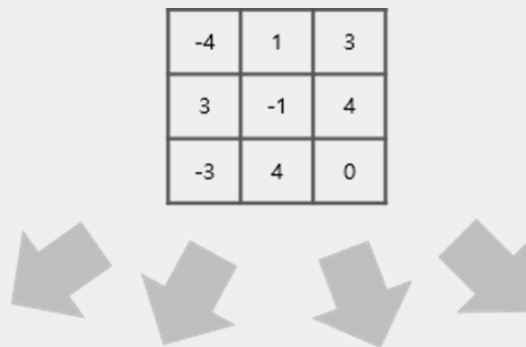
-4	1	3
3	-1	4
-3	2	0

3 × 3개의 숫자를 입력 받습니다.

선택된 3개의 숫자의 곱이 MAX인 값을 찾고,

MAX가 되는 수가 총 몇 가지 조합(중복 포함)이 있는지 출력 해 주세요.

아래 예제에서는 MAX값이 나오는 조합이 총 4개 있습니다.



-4	1	3
3	-1	4
-3	4	0

-4	1	3
3	-1	4
-3	4	0

-4	1	3
3	-1	4
-3	4	0

-4	1	3
3	-1	4
-3	4	0

[예제]

아래 예제에서는 총 7가지 조합이 있습니다.

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

3	-3	5
3	-3	5
1	-5	3

입력 예제

-4 1 3

3 -1 4

-3 4 0

출력 결과



문제 2번

뒷산에서 수 많은 한약 재료들을 배열에 담아왔습니다.

최대 20자리의 대문자로 구성된 문자열을 입력받아주세요.

이곳에 쓰인 알파벳(재료)들을 선정해서 **세 자리로 구성된** 새로운 조합(한약)을 만들어보려고 합니다.

사전 순으로, 중복없이 만들어지는 모든 조합을 출력 해 주세요.

입력 예제

ZFAABABBAB

출력 결과

AAA
AAB
AAF
AAZ
ABB
ABF
ABZ
AFF
AFZ
AZZ
BBB
BBF
BBZ
BFF
BFZ
BZZ
FFF
FFZ
FZZ
ZZZ



문제 3번

음료	금액	포인트
 아메리카노	500원	30
 카페라떼	300원	40
 모히또	700원	10
 콜라	400원	20
 와인	600원	30

위 메뉴에서 5개의 음료 중 n 개를 골라, 10,000원 어치 구매하려고 합니다.
 n 개를 항상 같은 수량으로 최대한 구매해야 한다면 얻을 수 있는 MAX 포인트는 얼마일까요?

예를들어

만약 $n = 2$ 이고, 아메리카노(500원, 30p) 모히또(700원, 10p) 선택해서
 10,000원 어치 최대한 구매하면

얻을 수 있는 포인트 값은 아래와 같습니다.

1,200원 $\times$ 8 = 9,600원 | 40p $\times$ 8 = 320 포인트 입니다.

입력 예제

4

출력 결과

600



문제 4번

디아블로 퍼즐이 있습니다. 이 퍼즐을 풀면 악마가 깨어납니다.

아래와 같이 Z 기준으로 한번 돌리면 Z를 둘러싸고 있는 문자들이 회전합니다.



가로로 AAA가 맞춰지면 퍼즐이 맞춰진 것입니다.

퍼즐 상태를 입력받고

정확히 6회 안에(6회 포함) 퍼즐을 맞추 수 있는지 판단하는 프로그램을 작성해주세요.

맞출 수 있으면 가능, 그렇지 않으면 불가능이라고 출력 합니다.

예를 들어

아래 예제에서는 ①~④ 번 까지 선택 할 수 있고, 선택하면 퍼즐이 시계방향으로 돌아갑니다.

A	B	T	B
A	① T	② C	D
B	③ G	④ A	D
A	B	C	T

② 를 한번 돌립니다. ③ 을 한번 돌립니다. ② 를 한번 돌립니다.



3번 만에 가로로 AAA가 맞춰졌습니다. 디아블로가 깨어날 가능성이 있습니다.



입력 예제

ABTB

ATCD

BGAD

ABCT

출력 결과

가능



문제 5번

시어머니와 며느리는 명절을 맞아 고기를 굽고 있습니다.

O는 앞면, X는 뒷면입니다.

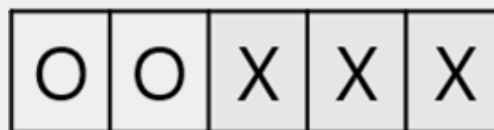
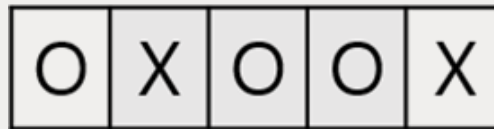
고기의 상태를 입력받습니다.



시어머니가 고기를 뒤집을 곳을 지목합니다.

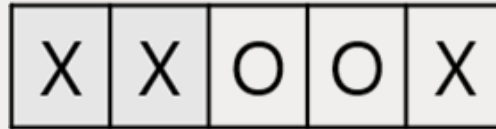
며느리는 지목 당한 곳과 양쪽 좌우를 모두 뒤집습니다.

아래 예제는 총 두번만에 고기를 모두 같은 방향으로 고기를 뒤집을 수 있습니다.



만약 시어머니가 가장자리를 선택했다면,

며느리는 2개의 고기만 뒤집습니다.



시어머니와 며느리는 최소 몇 번 고기를 뒤집어서,
모두 같은 방향 (전체 OOOO... 또는 전체 XXXX...) 으로 만들 수 있을까요?
모두 같은 방향으로 뒤집는 최소 횟수를 출력 해 주세요.

만약 4회 만에 뒤집는 것을 실패한다면 impossible 이라고 출력 해 주세요.

[세부조건]

고기의 개수는 최대 20개 까지 될 수 있습니다.

입력 예제

OXOOX

출력 결과

2